Weiler–Atherton polygon clipping algorithm

Dimitrije Radjenović
Faculty of Mathematics, University of Belgrade

Abstract

Polygon clipping, or computing the intersection of two planar polygons, is a core operation in computational geometry, essential for rendering, vector graphics, and spatial analysis. This work presents an implementation of the Weiler–Atherton algorithm capable of handling arbitrary simple polygons (without self-intersections and holes), including concave shapes and multiple disjoint output regions. The algorithm classifies intersection points as entry or exit nodes to direct topological traversal along polygon edges. The approach maintains ordered vertex lists with inserted intersection points. Validation on different examples, supported by SFML-based visualization, confirms numerical accuracy, topological consistency, and $O(nm)$ time complexity. Thus, the implementation offers a reliable foundation for Boolean operations on polygons.

Contents

1 Introduction **2**

2 Problem description **3**
2.1 Polygons . . . 3
2.2 Polygon intersection . . . 3

3 Weiler–Atherton clipping algorithm **3**
3.1 Traversal decision rules . . . 4
3.2 Step-by-step walkthrough . . . 5

4 Implementation **6**
4.1 Data Structures . . . 6
4.2 Geometric utilities . . . 6
4.3 Algorithm execution flow . . . 7
4.4 Edge cases and robustness . . . 7
4.5 Visualization and Input/Output . . . 7
4.6 Implementation notes . . . 8

5 Conclusion **8**

1 Introduction

Computing planar polygon intersections, commonly referred to in the literature as polygon clipping, constitutes a fundamental problem in computational geometry with broad applicability [1,2]. In computer graphics, clipping algorithms optimize scene rendering and shadow processing, thereby reducing the number of objects requiring rasterization [3]. Vector graphics systems, such as Inkscape, rely on these operations to execute Boolean operations on shapes (union, intersection, difference), while geographic information systems (GIS) employ intersection algorithms to analyze overlaps of land parcels and spatial datasets [4].

Although the geometric definition of polygon intersection is unambiguous, algorithmic implementation varies according to the topological properties of the input data. For convex polygons, the problem is well-established and solvable in linear time. The algorithm proposed by O'Rourke et al. (1982) achieves an asymptotic time complexity of $O(n+m)$, where n and m denote the number of vertices of the two polygons [5]. Due to convexity, at most a single connected intersection region is guaranteed. The general case, encompassing concave polygons with holes and the potential for multiple disjoint intersection regions, demands more sophisticated methodologies. Several strategies have been developed to address these challenges, ranging from sweep-line techniques to topological decomposition methods [6]. Among these distinct algorithmic frameworks, the Weiler–Atherton algorithm, introduced in 1977, remains a prominent solution for general polygon clipping with worst-case time complexity $O(nm)$ [7].

The Weiler–Atherton algorithm relies on topological traversal rather than simple geometric testing, classifying intersection points as entry or exit nodes to clip polygon boundaries. [7]. This approach naturally handles complex configurations, such as multiple disjoint intersection regions, setting it apart from simpler clipping methods. While the Greiner–Hormann algorithm [8] represents a later refinement with comparable robustness, the Weiler—Atherton algorithm remains the cornerstone of polygon clipping. Its boundary-switching paradigm established the foundational framework for topological traversal methods and continues to serve as the pedagogical reference for understanding complex polygon interactions.

This work presents a C++ implementation of the Weiler–Atherton algorithm [7], enhanced with numerical robustness techniques to address practical computational challenges. The implementation maintains synchronized ordered vertex lists for both the subject and clipping polygons, inserting computed intersection points at their precise geometric positions within the sequences. Developed with SFML [9] for visualization, the implementation focuses on arbitrary simple polygons without holes or self-intersections. Tests on different examples with SFML-based visualization confirm numerical accuracy, topological consistency, and $O(nm)$ time complexity. Thus, this work provides a solid foundation for Boolean operations in vector graphics and spatial analysis systems [7, 8].

2 Problem description

2.1 Polygons

Geometrically, a polygon P defines a closed region in the Euclidean plane $\mathbb{R}^2$, bounded by a finite chain of line segments forming a simple closed curve, denoted as the boundary ∂P [2]. Computationally, P is represented by an ordered sequence of vertices $V = \{v_1, v_2, \ldots, v_n\}$. The boundary consists of edges e_i connecting consecutive vertices (v_i, v_{i+1}), with the final edge closing the loop (v_n, v_1).

A polygon is termed *simple* if its boundary is non-self-intersecting [2]. Formally, adjacent edges share exactly one common vertex, while non-adjacent edges share no points. For the purpose of clipping algorithms, the vertex sequence assumes a consistent orientation, which defines the interior of the polygon via the winding number rule.

2.2 Polygon intersection

Given two polygons P (*subject polygon*) and Q (*clipping polygon*), the *intersection* $P \cap Q$ is defined as the set of all points that lie simultaneously in the interior or on the boundary of both polygons:

$$P \cap Q = \{x \in \mathbb{R}^2 \mid x \in P \wedge x \in Q\}. \tag{1}$$

The boundary of $P \cap Q$ consists of alternating subsequences of edges from ∂P and ∂Q, connected at *intersection points* where edges from the two polygons cross. Unlike the intersection of convex polygons, which always yields a single connected convex region, the intersection of concave polygons may produce multiple disjoint connected components, as illustrated in Figure 1.
Note that in computer graphics literature, this operation is frequently referred to as general polygon *clipping*, where Q represents the clipping region [3, 7].

Objective. The primary goal of this work is to compute the boundary representation of all connected components resulting from the intersection $P \cap Q$ through an implementation of the Weiler-Atherton algorithm. This method is selected for its robustness in handling concave polygons and multiple disjoint outputs while presuming correct vertex ordering and orientation. The scope is limited to arbitrary simple polygons without holes.

3 Weiler–Atherton clipping algorithm

The Weiler–Atherton algorithm clips arbitrary subject polygons against a clipping polygon by traversing both boundaries and switching at intersection points to generate closed output regions [7]. The algorithm proceeds through the following steps:

1. Compute all intersection points between the edges of the subject polygon and the clipping polygon.

2. Classify each intersection point as either entering or exiting based on the traversal direction of the subject polygon relative to the clipping region.

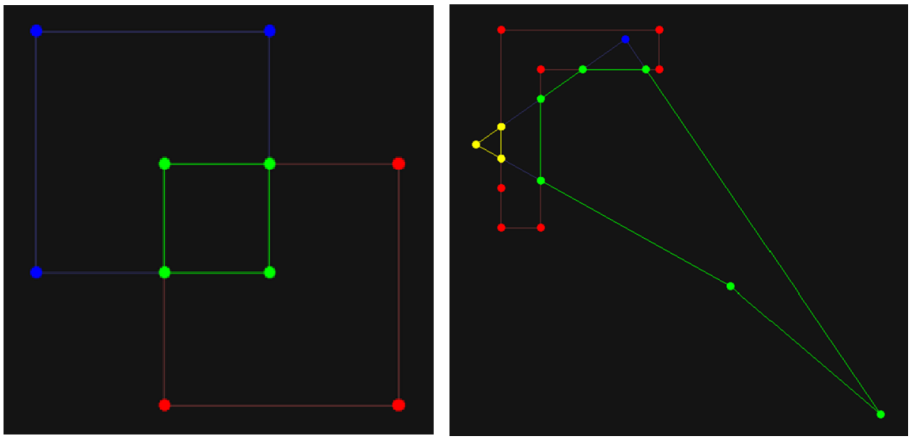

Figure 1: Polygon clipping comparison: convex polygons (left) yield a single intersection region (colored in green), whereas concave polygons (right) may produce multiple disjoint regions (colored in green and yellow).

3. Initialize two ordered vertex lists, one for the subject polygon and one for the clipping polygon.
4. Insert the intersection points into both lists at their correct spatial positions between the original vertices and establish bidirectional links between corresponding intersections.
5. Select the first unvisited entering intersection point from the subject polygon list to begin the clipping process.
6. Traverse the subject polygon list sequentially from the selected point, appending each vertex to the output vector until an exiting intersection point is encountered.
7. Switch traversal to the clipping polygon list via the linked exiting intersection point. Continue traversing and appending vertices until the original entering intersection point is reached, thereby closing the polygon loop.
8. Store the accumulated vertex vector as a resulting clipped polygon. This ensures that multiple disjoint polygons are correctly captured in cases involving concave geometries.
9. Repeat the traversal procedure starting from step 5 for all remaining unvisited entering intersection points. Terminate the algorithm when every entering intersection has been processed exactly once.

3.1 Traversal decision rules

The core traversal mechanism relies on the following decision rules to ensure topological consistency:

- **Entry points:** Detect intersection, record it, and continue on subject polygon.

- **Exit points:** Detect intersection, record it, and switch to clip polygon boundary.
- **Inside vertices:** Record all vertices that lie within the clip region.
- **Outside vertices:** Do not record; skip until next intersection.
- **Direction:** Always traverse both polygons in clockwise direction.

3.2 Step-by-step walkthrough

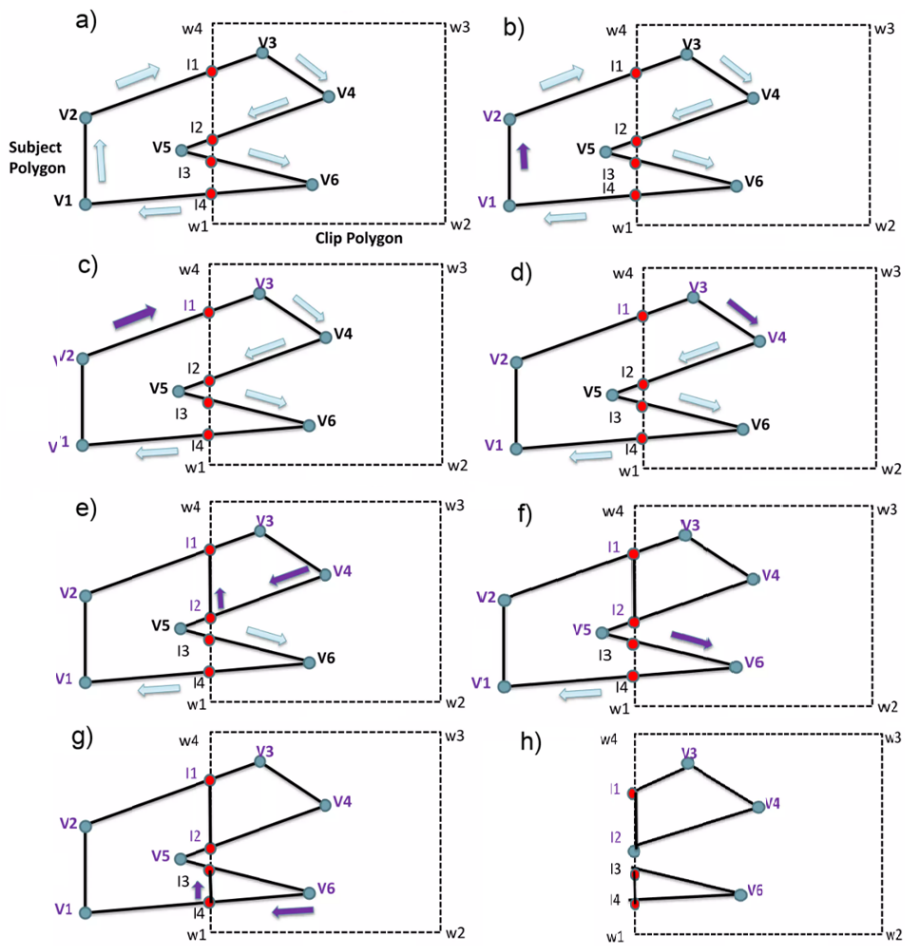

Figure 2: Clipping sequence overview: a) Initial setup, b-g) Tracing through vertices and switching at intersections, h) Final clipped polygon.

Figure 2 illustrates the step-by-step execution of the Weiler-Atherton algorithm:

a) **Setup:** Calculate all intersection points (I1–I4) between subject and clip polygon boundaries. Classify each as *Entry point* (transition from outside to inside) or *Exit point* (transition from inside to outside).

b) **Start traversal:** Begin at vertex V1 and traverse the subject polygon clockwise. Move from V1→V2; since both vertices are outside the clip region, no recording is performed.

c) **Entry point detected (I1):** When traversing from V2→V3, an intersection with the clip boundary is detected at I1. Since the transition is from outside to inside, I1 is marked as an Entry point. Record I1 and continue following the subject polygon boundary to V3.

d) **Inside region:** Continue from V3→V4; both vertices lie inside the clip region. Record vertices V3 and V4. Proceed until Exit point I2 is detected (transition from inside to outside).

e) **Switch to clip polygon:** At Exit point I2, stop following the subject polygon. Switch to the clip polygon boundary and follow it clockwise. Record the clip boundary segment from I2 to the next intersection point I3.

f) **Re-enter subject polygon:** At I3, an Entry point is detected (transition back to subject polygon). Switch from clip boundary to subject polygon. Record I3 and continue traversing the subject polygon through V5 to V6.

g) **Exit and close loop:** At V6, detect Exit point I4. Switch back to clip polygon boundary and follow it clockwise from I4 until reaching the starting point I1, thereby closing the clipped polygon loop.

h) **Final result:** The clipped polygon consists of: I1→V3→V4→I2→I3→V5→V6→I4→I1.

4 Implementation

The Weiler–Atherton algorithm is implemented in C++ using the SFML library [9] for visualization. The code prioritizes clarity and direct translation of the algorithmic steps over abstract architectural patterns.

4.1 Data Structures

- `Point`: Represents a 2D coordinate with floating-point precision. Equality comparison uses epsilon tolerance (`EPS = 1e-6`) to handle numerical instability.
- `Node`: Extended vertex structure containing:
  - Geometric coordinates (`point`).
  - Type flag (`VERTEX` or `INTERSECTION`).
  - Edge indices in subject/clip polygons.
  - `isEntry` flag for traversal direction.
- `class Polygon`: Container for vertex list with rendering methods (`draw()`, `drawPoints()`) for SFML integration.

4.2 Geometric utilities

All geometric predicates are implemented as free functions operating on plain data:

- `isPointInside()`: Ray-casting algorithm for point-in-polygon testing.
- `orientation()`: Cross-product based counter-clockwise test.

- `segmentsIntersect()`: Checks if segments intersect.
- `getIntersection()`: Gets segment intersection using Cramer's rule.

4.3 Algorithm execution flow

The main function `weilerAthertonClip()` orchestrates the following procedural pipeline:

1. **Intersection detection:** `FindIntersections()` performs brute-force $O(n \cdot m)$ pairwise edge testing between subject and clip polygons.
2. **Insertion and ordering of intersection nodes:** `addIntersections()` inserts intersection nodes into vectors, sorted by distance from vertex nodes.
3. **Entry/Exit labeling:** `classifyIntersections()` performs a single linear pass over the extended subject polygon, toggling an inside/outside state flag to classify each intersection as Entry or Exit.
4. **Flag synchronization:** `matchIntersectionFlags()` propagates classification results to the clip polygon's intersection nodes.
5. **Result construction:** `createPolygons()` implements the core traversal logic:
   - Locate first unprocessed Entry point in subject polygon.
   - Follow current boundary clockwise, appending vertices to output.
   - Upon reaching an Exit point, switch to the corresponding node in the other polygon.
   - Continue traversal, switching at each intersection, until returning to start.
   - Repeat for remaining Entry points to handle multiple disjoint output polygons.

4.4 Edge cases and robustness

- **Non-intersecting polygons:** Early exit checks containment of a single subject vertex; returns either the full polygon or empty result.
- **Duplicate vertex cleanup:** Post-processing pass removes consecutive duplicate coordinates and redundant closing vertices.
- **Floating-point tolerance:** All geometric comparisons use epsilon thresholds to mitigate precision errors in intersection calculations and equality tests.

4.5 Visualization and Input/Output

- **SFML rendering:** Original polygons are drawn semi-transparently (red/blue); clipped results use distinct colors (green/yellow/magenta/cyan) with thicker outlines for clarity (see Figure 3).
- **File input:** Polygons can be loaded from text files (`polygon.txt`, `clip.txt`) with support for comments (`#`) and fallback to default test cases.
- **Debug display:** A window that shows information about clipped polygons and their construction.

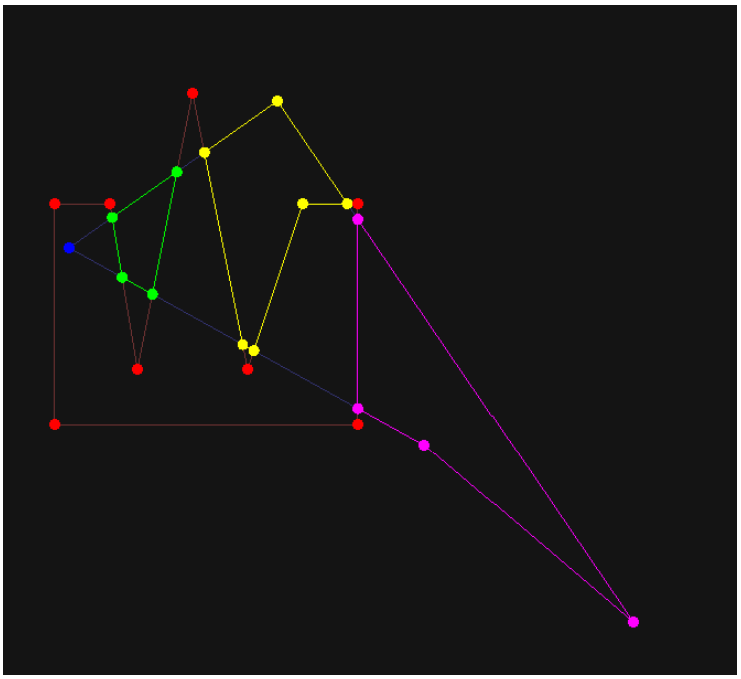

Figure 3: Weiler-Atherton algorithm result: clip polygon (red), subject polygon (yellow), intersection points (green), and final clipped output (magenta). Generated using the SFML-based implementation.

4.6 Implementation notes

- No external dependencies beyond SFML and C++ standard libraries.
- Time complexity is dominated by intersection detection: $O(nm)$ for n subject and m clip edges. Traversal is linear in the number of extended vertices.

5 Conclusion

This work confirms that the algorithm, formalized by Weiler and Atherton [7], remains a viable solution for clipping arbitrary simple polygons (without holes or self-intersections). By utilizing a topological traversal structure and resolving intersections, the system successfully constructs clipped polygons, even when resulting in multiple disjoint components. The code is written in C++ and supports loading polygons from text files. Performance aligns with the theoretical worst-case bound for boundary traversal methods. Testing within a display environment confirms both the numerical accuracy and topological consistency of the generated results. Performance analysis verifies the expected $O(nm)$ time complexity, establishing this implementation as a reliable basis for Boolean operations in vector graphics systems such as Inkscape.

References

[1] J. O'Rourke, *Computational Geometry in C.* (Cambridge University Press, 1998).

[2] M. de Berg, O. Cheong, M. van Kreveld, and M. Overmars, *Computational Geometry: Algorithms and Applications.* (Springer Berlin, 3rd edition, 2008).

[3] A. van Dam, S.K. Feiner, J.F. Hughes, and J.D. Foley, *Computer Graphics: Principles and Practice.* (Addison-Wesley, 2nd edition, 1995).

[4] P.A. Longley, M.F. Goodchild, D.J. Maguire, and D.W. Rhind, *Geographic Information Science and Systems.* (John Wiley & Sons, 2015).

[5] J. O'Rourke, C.B. Chien, T. Olson, and D. Naddor, A new linear algorithm for intersecting convex polygons, *Computer Graphics and Image Processing*, **19**, 384–391, 1982.

[6] J.L. Bentley and T.A. Ottmann, Algorithms for reporting and counting geometric intersections, *IEEE Transactions on Computers*, **C-28**, 643–647, 1979.

[7] K. Weiler and P. Atherton, Hidden surface removal using polygon area sorting, *ACM SIGGRAPH Computer Graphics*, **11**, 214–222, 1977.

[8] G. Greiner and K. Hormann, Efficient Clipping of Arbitrary Polygons, *ACM Transactions on Graphics*, **17**, 71–83, 1998.

[9] SFML – Simple and Fast Multimedia Library. Available at: https://www.sfml-dev.org/.